这些内容放到线上，是因为它们以后会在真实项目里反复回来找你。等你真的要连公司 GitLab、推实验分支、解决冲突、排查 clone 失败、避免把 checkpoint 或密钥提交上去时，再扫码看完整版，会比现在把一堆命令硬塞进纸书里更有用。

1.7.7 自检清单

1. Git 和 GitHub 的区别是什么？如果你在一台断网的服务器上，还能使用 Git 吗？
   可以。Git 是安装在你本地的工具，提交、查看历史这些核心操作不需要网络。GitHub/GitLab 是远程同步平台。
2. `git add` 和 `git commit` 分别在做什么？
   `git add` 把改动放进暂存区，也就是给这次要拍快照的内容打勾；`git commit` 把暂存区里的内容存成一张本地快照。
3. `git push` 和 `git commit` 的区别是什么？
   `commit` 是把快照存在本地仓库；`push` 是把本地新增的提交同步到远程仓库，让团队也能看到。
4. 为什么建议经常敲 `git status`？
   因为它能告诉你哪些文件改了、哪些已经暂存、有没有奇怪文件混进来。新手很多 Git 事故都可以靠多看 `status` 提前避免。
5. 哪些东西不应该提交到 Git 仓库？
   模型权重、数据集、训练日志、虚拟环境、本地配置，以及 API Key、密码、token 这类敏感信息。

1.7.8 Python 交互实验室

Git 的命令只看语法很容易混在一起，配套代码 `git_workflow_lab.py` 会在你的机器上自动搭建一个临时仓库，先带你走一遍纸质版讲过的基本流程：查看状态、暂存改动、本地提交。脚本后半段还会演示分支、冲突和 `.gitignore`，这些内容对应线上完整版。如果你现在只读纸质版，可以先看前半段，等扫码读完完整版后再回来跑完整流程。

建议你先读完纸质版，建立基本直觉；真正想把命令跑一遍时，再配合这个实验脚本一起练。看到终端输出之后，Git 会比纯文字好懂很多。

1.8 浮点数常识：计算机是怎么存小数的

1.8.1 一个你迟早会遇到的场景

你花了三天时间准备数据、写训练脚本、调参数，终于把模型跑起来了。前几百步一切正常，loss 稳稳地往下降，你甚至开始期待最终的效果。然后你去倒了杯咖啡回来，屏幕上的 loss 突然变成了一个单词：`nan`。

nan，不是一个数字（Not a Number）。你的模型“炸了”。

你上网搜，有人说“调小学习率”，有人说“换一种数据精度”，有人说“加梯度裁剪”。你照做了，有时候好了，有时候没好，但你始终不太确定：这些方法为什么有用？或者说，这东西到底为什么会炸？

答案藏在一个非常基础、但很少有人认真给你讲清楚的地方：计算机是怎么存储小数的。

这一节讲的就是这件事。我要提前告诉你：这一节的篇幅会比你预想的长，我会讲得非常细，细到带你亲手把一个十进制数字拆成一串 0 和 1，再把这串 0 和 1 还原回来。篇幅长，是因为浮点数是大模型工程领域里被严重低估的知识点，我个人认为没有之一。

为什么这么说？因为你搞清楚浮点数之后，会一下子建立起很多直觉。你会知道你训练的模型，那几十亿个参数，在显存里到底是以什么形态存在的。你会明白为什么同一个模型用不同的“精度”加载，占用的存储空间差了一倍。你会理解训练“炸了”的时候到底发生了什么。而且当你后面学到模型量化（就是把一个训练好的大模型压缩到更小的体积、让它能在更便宜的设备上跑起来）的时候，这里建立的直觉会让你学起来非常顺畅。

1.8.2 计算机为什么存不了 0.1

在你的直觉里，0.1 是一个再简单不过的数字。你写`x = 0.1`，你觉得计算机就老老实实地存了一个 0.1。

但事实不是这样。你现在打开 Python，输入以下内容：

```
print(0.1 + 0.2)
```

输出结果不是 0.3，而是：

```
0.30000000000000004
```

第一次看到这个结果的人，通常会怀疑是不是 Python 有 bug。但这不是 Python 的问题，用 C、Java、JavaScript，结果都一样。这是所有计算机在存储小数时的共同特征。

要理解为什么，你需要先接受一个前提：计算机内部只有两种状态——通电和断电，对应数字 1 和 0。所有的数据——文字、图片、视频、你的模型参数——在计算机内部都是用一串 0 和 1 来表示的。这套只用 0 和 1 的计数方式叫做二进制。

计算机里存储一个 0 或者一个 1，占据的存储空间叫做一个**位**（bit）。位是计算机世界里最小的存储单位。但一个位只能表示 0 或 1，实在太小了，所以人们约定把 8 个位凑在一起，作为一个更常用的单位，叫做一个**字节**（byte，注意 byte 和 bit 只差一个字母，但含义完全不同）。你平时说“这个文件 8MB”，那个 B 就是字节。“字节跳动”那个“字节”，指的也是它。

位和字节之间的关系很简单：1 字节 = 8 位。字节往上的换算稍微特殊一点——在计算机存储领域里，不是逢一千进一，而是逢 1024 进一。为什么是 1024？因为 1024 恰好是 2^{10}，和二进制的体系更契合。

单位	全称	换算关系
B（Byte）	字节	= 8 个位（bit）
KB	千字节（Kilobyte）	= 1024 B
MB	兆字节（Megabyte）	= 1024 KB
GB	吉字节（Gigabyte）	= 1024 MB

日常使用中，你可以把 1024 粗略当成 1000 来估算。比如 1GB 大约是 10 亿字节，1MB 大约是 100 万字节。后面讲模型大小的时候我们会用到这张表，到时候你会发现这个换算关系一点都不抽象。

你平时用的是十进制，逢十进一。二进制是逢二进一。比如十进制的 5，用二进制写就是 101（一个 4 加一个 1）。整数转换成二进制，总能精确表示，只要位数够多就行。

但小数就不一样了。

你有没有试过用十进制去写 1/3？0.333333……永远写不完。十进制这套体系天然没办法用有限的位数精确表示 1/3。

二进制也有同样的问题，只不过它“写不完”的数不一样。

你可以回想一下，十进制的小数是怎么拼出来的？比如 0.125，它是由 1 个 0.1（十分之一），2 个 0.01（百分之一）和 5 个 0.001（千分之一）拼起来的。

二进制也一样，只不过它的“积木”变了。因为逢二进一，二进制小数的积木是 1/2（也就是 0.5）、1/4（0.25）、1/8（0.125）、1/16（0.0625）……所有的积木都是“2 的某次方分之一”。

那么问题来了：在二进制的世界里，你要怎么用这些积木拼凑出十进制的 0.1？

你可以试着在脑海里拼一下：0.1 比 0.5、0.25、0.125 都要小，所以前几块大积木都用不上。你能塞进去的第一块积木是 1/16（0.0625），放进去之后，距离 0.1 还差 0.0375。接下来，你只能继续找更小、更碎的积木（1/32、1/64……）往里填补。

如果你一直拼下去就会发现一个有点沮丧的事实：无论你往下切分多少次，把这些“2 的倍数分之一”的积木加起来，你只能无限地逼近 0.1，却**永远无法严丝合缝地凑出正好的 0.1**。

十进制里的 0.1，在二进制的世界里恰好就是一个**无限循环小数**。所以当你写`x = 0.1`的时候，计算机并没有存下精确的 0.1。它存的是一个非常接近 0.1、但不完全等于 0.1 的值。两个这样的近似值加在一起，误差就暴露出来了。

这不是一个 bug，这是二进制在数学层面的固有限制。你没有办法“修复”它，只能理解它、然后在工程中正确地应对它。

1.8.3 用有限的格子去装无限的数

既然计算机只能用有限位数的 0 和 1 来存数字，那它面临的核心问题就是：实数有无穷多个，而我只有有限个格子，怎么办？

我们先看一个你熟悉的例子。在科学课本上，很大的数字或很小的数字通常不会老老实实地写出来，而是写成科学计数法。比如光速大约是 300000000 米每秒，写成科学计数法就是 3.0×10^8。地球到太阳的距离大约是 149600000000 米，写成 1.496×10^{11}。这样做的好处是：不管数字有多大或多小，你只需要记录两件事，有效数字（3.0、1.496）和数量级（10 的几次方）。

计算机存小数用的办法，本质上就是二进制版本的科学计数法。

一个小数在计算机内部，会被表示成这样的形式：

$$\pm 1.xxxxx \times 2^n$$

其中：

- 最前面的 ± 决定这个数是正还是负。

- 1.xxxxx 是有效数字部分，在计算机里叫做**尾数**（mantissa），决定了这个数的精度，也就是小数点后面能保留多少位。
- 2^n 中的 n 是指数（exponent），决定了小数点要往左还是往右“飘”多远。它控制的是这个数的大小范围。

“浮点数”这个名字就是从这里来的：小数点的位置不是固定的，而是根据指数的大小“浮动”的。指数大，小数点就飘到右边，表示一个很大的数；指数小（甚至是负数），小数点就飘到左边，表示一个很小的数。

所以，在计算机内部存储一个小数，就是把有限的二进制位分成三份：一位给符号，一部分给指数，剩下的给尾数。

这就引出一个关键的权衡：总共就这么多位，分给指数的多，能表示的数字范围就大（能存很大的数，也能存很小的数），但分给尾数的就少了，精度就差；反过来，分给尾数的多，精度高了，但指数位少了，能表示的范围就小。

这个权衡，是你理解后面所有内容的关键。

1.8.4 全世界的约定：三种最常见的浮点格式

你可能会想：这种“三份怎么分”的问题，不同的公司会不会有不同的分法？

早期确实是这样，各家各搞一套，数据一换平台就乱了。后来 IEEE（一个国际电气和电子工程领域的标准化组织）在 1985 年发布了一个标准，编号 754，统一了浮点数的存储方式。从此全世界的计算机在存小数这件事上，终于说上了同一种语言。

你不需要去读那份标准文档，你只需要记住下面这张表。这张表是你在大模型工作中每天都会用到的东西。

格式	总位数	符号	指数	尾数	每个数占多少字节	PyTorch 中的写法
FP32	32 位	1 位	8 位	23 位	4 字节	torch.float32
FP16	16 位	1 位	5 位	10 位	2 字节	torch.float16
BF16	16 位	1 位	8 位	7 位	2 字节	torch.bfloat16

“每个数占多少字节”这一列是从总位数直接算出来的：32 位 $\div$ 8 = 4 字节，16 位 $\div$ 8 = 2 字节。这个换算后面会反复用到。当你要算一个模型占多少存储空间的时候，就靠它。这三种格式具体是怎么把有限的位宽分给符号、指数和尾数的呢？你可以结合图 1.6 的位宽分配示意图来建立一个直观的画面。

先看 FP32，也叫单精度浮点数。8 位指数让它能表示的范围非常大，从大约 10^{-38} 到 10^{38}。23 位尾数给了它大约 7 位十进制有效数字的精度。在很长一段时间里，FP32 是深度学习训练的默认格式。

再看 FP16，半精度浮点数。每个数字只占 2 字节，存储空间是 FP32 的一半。但注意它的指数只有 5 位，这意味着它能表示的最大值只有大约 65504。六万多，听起来不小？但你想想，模型训练过程中，某些层的中间计算结果、梯度累加值或者经过放大处理后的数值超过六万是完全可能的事情。一旦超过，这个数就“溢出”了，变成无穷大。

最后是 BF16，全称是 Brain Floating Point（这个名字来自 Google Brain 团队，他们发明了这种格式）。它也是 16 位，和 FP16 一样省空间。但你仔细看表格，它的指数和 FP32 一样是 8 位，代价是把尾数从 FP16 的 10 位砍到了 7 位。

这意味着什么？BF16 的常用数值范围和 FP32 非常接近，不容易溢出；但它的精度比 FP16 还要差，只有大约 2 到 3 位十进制有效数字。

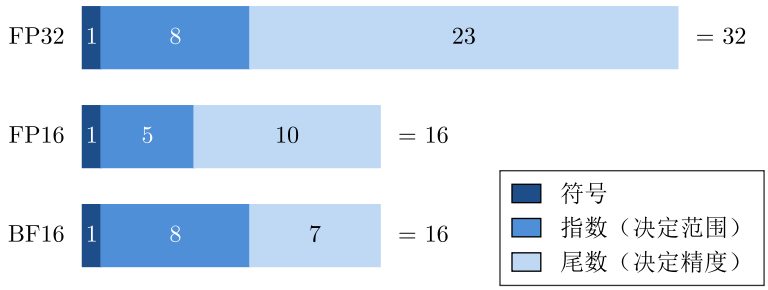


图 1.6: FP32、FP16 与 BF16 的位宽分配对比。每一格代表一个二进制位，条形总长度反映总位数。注意 BF16 的指数段与 FP32 等宽，均为 8 位，这意味着两者的数值范围非常接近。

BF16 的设计哲学可以用一句话概括：宁可看不清细节，也不能看不到全局。它主动牺牲了精度来保全范围。这个取舍在大模型训练中非常合理。神经网络的训练过程本身就充满随机性（每次随机采样一批数据、参数随机初始化），对精度的容忍度很高；但对范围的容忍度极低，因为一旦某个数溢出变成无穷大，整个训练就会像多米诺骨牌一样崩掉。

到今天，BF16 已经成为大模型预训练阶段事实上的标准格式。

1.8.5 一个模型到底有多大

学完三种浮点格式之后，我们来做一件马上就能用的事情：算一个大模型占多少存储空间。

你可能听过别人说“这是一个 8B 模型”“那是一个 70B 模型”。这个 B 是什么意思？

在大模型领域，描述参数量的时候用的是一套简写。这套简写和前面讲的存储单位（KB、MB、GB）长得有点像，但含义完全不同，千万不要搞混。

缩写	含义	具体数量
K	千（Kilo）	1,000
M	百万（Million）	1,000,000
B	十亿（Billion）	1,000,000,000

注意看，参数量这套缩写是严格按“逢千进一”来的：1B = 1000M，1M = 1000K。而前面讲的存储单位（KB、MB、GB）是逢 1024 进一。这两套体系的进制不一样，但在日常估算中，1024 和 1000 的差距不到 3%，不影响你做粗略判断。

所以当你听到“8B 模型”的时候，意思就是：这个模型有 8B 个参数，也就是 8,000,000,000 个——80 亿个。

那什么是“参数”？你现在不需要知道它的来龙去脉（后面讲模型训练的时候会详细说），只需要建立一个最基本的画面：一个大模型，说到底，就是一大堆数字。这些数字叫做参数，是模型通过训练“学”到的东西。你在网上下载一个模型文件，硬盘上占了多少空间，取决于两件事——这些数字有多少个，以及每个数字用什么格式存。

现在你已经知道了格式和参数量，可以直接算了。

一个 8B 模型有 8,000,000,000 个参数。如果每个参数用 FP32 存储，每个占 4 字节：

$$8,000,000,000 \times 4 = 32,000,000,000\text{字节} \approx 32\text{GB}$$

这里的 32GB 是工程里的粗略估算，默认把 1GB 近似看成 10 亿字节。严格按 1024 进制换算，数字会略小一点，但不影响我们判断模型大概需要多少空间。

如果换成 BF16，每个参数占 2 字节：

$$8,000,000,000 \times 2 = 16,000,000,000\text{字节} \approx 16\text{GB}$$

同样的模型，仅仅是换了一种存储格式，占用的空间就差了一倍。

这个计算虽然简单，但它是你日常工作中最常做的估算之一。以后你看到任何一个模型，只要知道参数量和存储格式，就能立刻算出它的大小。比如一个 70B 模型用 BF16 存储：70,000,000,000 × 2 = 140,000,000,000 字节 ≈ 140GB。

这里说的是模型文件存在硬盘上的大小。当你要把模型加载到 GPU 上跑起来的时候，它会被搬到显卡自带的一块存储空间里，叫做**显存**——关于显卡和显存，下一节会详细讲，你现在只需要知道它是一块容量有限的存储空间就行。如果显存不够大，模型就装不进去。而且训练的时候，显存里不只要放模型参数本身，还要放中间计算结果和梯度（这些概念后面会讲），实际占用会远大于模型参数本身的大小。

这也是为什么浮点格式的选择对大模型工程师来说不是一个理论问题，而是一个“装得下还是装不下”的现实问题。

1.8.6 亲手拆开一个浮点数

上面讲了浮点数的三个组成部分和三种常见格式，但到目前为止都还停留在“我知道有这么回事”的层面。接下来我要带你做一件事：亲手把一个十进制数字变成一串 32 位的 0 和 1，然后再把这串 0 和 1 还原回来。走完这个过程之后，IEEE 754 就不再是一张需要背诵的表格，而是一个你亲手拆装过的东西。

我们选一个简单的数字：6.75。

第一步：把十进制转成二进制。

先处理整数部分。6 用二进制怎么写？我们可以用十进制的常识来类比：

在十进制中，逢十进一，每一位都代表“10 的几次方”。比如数字 420，实际上是：

$$420 = 4 \times 10^2 + 2 \times 10^1 + 0 \times 10^0$$

在二进制中，逢二进一，每一位代表的则是“2 的几次方”。从右往左数：

- 第一位（个位）代表 2^0（也就是 1）
- 第二位代表 2^1（也就是 2）
- 第三位代表 2^2（也就是 4）

因为 $6 = 4 + 2 + 0$，我们可以把它写成：

$$1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

把这些对应位置上的系数（1，1，0）按顺序拼起来，就是二进制的 110。

再处理小数部分。0.75 用二进制怎么写？$0.75 = 0.5 + 0.25 = 2^{-1} + 2^{-2}$，所以是.11。

合在一起，6.75 的二进制是 110.11。

第二步：写成二进制的科学计数法。

就像十进制的 675.0 可以写成 6.75×10^2 一样，二进制的 110.11 可以写成 1.1011×2^2。小数点往左移了两位，所以指数是 2。

这里有一个重要的细节：在二进制科学计数法中，**只要数字本身不是 0，小数点前面的那一位永远是 1**。

我们可以分两种情况来看：

- **情况一：数字大于等于 1（小数点往左移）**。比如我们刚才的 6.75，二进制是 110.11。为了让小数点前面只有一个 1，你需要把小数点向**左**移 2 位，变成 1.1011×2^2。
- **情况二：数字小于 1（小数点往右移）**。假设你的数字是十进制的 0.375，转成二进制是 0.011。此时小数点前面是 0。因为二进制只有 0 和 1，只要这个数本身不是 0，你总可以把小数点一直向**右**移，直到碰到第一个 1 为止。把 0.011 的小数点向右移 2 位，它就变成了 1.1×2^{-2}。

所以，不管原数字长什么样，它的二进制科学计数法最终一定能被调整成 $1.xxxxx$ 的形式。

既然这个 1 永远在那里，计算机就做了一个聪明的决定：不存它。反正大家都知道那里有个 1，何必浪费一个位的空间？这个被省掉的 1 叫做“隐含的前导 1”。这意味着 23 位的尾数实际上能表示 24 位有效数字的精度，白赚了一位。

第三步：拆成三个部分，拼成 32 位。

现在我们把 1.1011×2^2 按照 FP32 的格式来编码：

符号位：6.75 是正数，符号位是 0。

指数部分：实际指数是 2。IEEE 754 存储指数时，会先加上一个固定的偏移量 127，所以这里存储的值是 $2 + 127 = 129$。为什么要加 127？因为指数需要能表示负数（比如 0.001 这种很小的数，指数是负的），但计算机用无符号整数来存储指数部分会更方便。FP32 的指数位有 8 位，一共能表示 0 到 255 这 256 个整数，127 差不多在中间，用它做偏移，就能把正指数和负指数都塞进同一套无符号编码里。加上 127 之后，实际指数为-126 的时候存储值是 1，实际指数为 +127 的时候存储值是 254，全都是正数，处理起来更简单。129 的二进制是 10000001。

尾数部分：1.1011 中，去掉隐含的那个 1，剩下的小数部分是 1011。FP32 的尾数有 23 位，所以在 1011 后面补上 19 个 0，变成 10110000000000000000000。

把这三部分拼在一起：

```
0 10000001 10110000000000000000000
```

去掉空格，就是 6.75 在计算机内存里真正的样子：

```
01000000110110000000000000000000
```

32 个 0 和 1，不多不少。

反过来：从 32 位还原回十进制。

现在假设你拿到了另一串 32 位的二进制：00111110001000000000000000000000。

拆成三部分:符号位是 0(正数),指数部分是 01111100,尾数部分是 01000000000000000000000。

指数部分 01111100 转成十进制是 124，减去偏移量 127，实际指数是-3。

尾数部分，我们在前面加上那个“隐含的前导 1”，它就变成了二进制的 1.01（后面长串的 0 没有实际意义，可以直接忽略）。

那二进制的 1.01 转换成十进制是多少呢？这里我们需要用到和小数点前类似的“位权”概念，只不过小数点后的每一位代表的是“2 的负几次方”：

- 小数点后第一位代表 2^{-1}（也就是 1/2，即 0.5）
- 小数点后第二位代表 2^{-2}（也就是 1/4，即 0.25）

所以，二进制的 1.01 用十进制表示就是：

$$1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 1 + 0 + 0.25 = 1.25$$

代入公式：$(-1)^0 \times 2^{-3} \times 1.25 = 1 \times 0.125 \times 1.25 = 0.15625$。

你可以用 Python 验证一下：

```
import numpy as np
print(np.float32(0.15625))
```

输出就是 0.15625，分毫不差。因为 0.15625 恰好能被二进制精确表示（它等于 1/8 + 1/32），所以没有任何舍入误差。

到这里，你已经完整地走过了一个浮点数的编码和解码全过程。接下来我要用这个基础，带你看一个更重要的东西。

1.8.7 给二进制串加一：亲眼看见精度的边界

你已经知道 6.75 的 32 位表示是 01000000110110000000000000000000。

接下来想一个问题：在 FP32 能表示的所有数字里，比 6.75 大的、离 6.75 最近的那个数是几？

这个数一定存在——FP32 能表示的数字是有限个的，所以每个数都有一个“下一个邻居”。找到这个邻居的方法非常简单：把 6.75 的 32 位表示在末尾加 1。

```
01000000110110000000000000000000 → 6.75
01000000110110000000000000000001 → ?
```

末尾加 1 之后,尾数部分从 10110000000000000000000 变成了 10110000000000000000001。这个新的 32 位串对应的十进制数字是多少？我们就不手算了。你可以用 Python 来算：

```
import numpy as np
a = np.float32(6.75)
b = np.nextafter(a, np.float32(np.inf))
print(b)
print(b - a)
```

第一行创建了 FP32 格式的 6.75。第二行用了一个函数`np.nextafter`，它的作用就是“告诉你 a 的下一个可表示的浮点数是几”。第二个参数`np.inf`表示往正方向找。

输出是：

```
6.7500005
4.7683716e-07
```

6.75 的“下一个邻居”是 6.7500005（更精确地说是 6.7500004768371 58）。它们之间的距离大约是 0.00000048。这意味着在 FP32 的世界里，6.75 和 6.7500005 之间不存在任何其他数字。如果你试图存一个 6.75000001，计算机会把它舍入成 6.75；如果你试图存一个 6.75000035，计算机也会把它舍入到 6.75 或 6.7500005 中更近的那一个。

你可以用代码直接验证这种舍入行为：

```
import numpy as np
f1 = np.float32(6.7500004)
f2 = np.float32(6.7500005)
f3 = np.float32(6.7500008)
print(f1 == f2)
print(f2 == f3)
```

第一个输出是`True`——6.7500004 和 6.7500005 在 FP32 看来是同一个数，因为它们都落在同一个格点上。第二个输出是`False`——6.7500005 和 6.7500008 被分到了不同的格点上，FP32 能区分它们。

从这个例子来看，在 6.75 这个位置上，FP32 能区分的最小差异出现在小数点后第 6 到第 7 位。这就是“FP32 有大约 6 到 7 位十进制有效数字”这个说法的来源。

但这个边界不是固定的。把同样的操作换到一个大数上。我们看看 10000000（一千万）在 FP32 里的邻居间距是多少：

```
import numpy as np
a = np.float32(10000000)
b = np.nextafter(a, np.float32(np.inf))
print(b)
print(b - a)
```

输出：

```
10000001.0
1.0
```

一千万的下一个可表示的浮点数，是一千万零一。间距是 1。这意味着在一千万附近，FP32 连小数都没有了——小数点后面所有的信息全部丢失，整数之间的精度也只勉强保住了。

如果数字继续变大呢？我们试试 123456789（大约 1.23 亿）：

```
import numpy as np
a = np.float32(123456789)
b = np.nextafter(a, np.float32(np.inf))
print(a)
print(b)
print(b - a)
```

输出：

```
123456792.0
```

```
123456800.0
8.0
```

注意看，123456789 存进 FP32 之后变成了 123456792——输入的数字本身就已经被舍入了。而它的下一个邻居是 123456800，间距是 8。在这个量级上，FP32 连相邻两个整数都区分不了了，中间一口气吞掉了好几个整数。

你可以自己用代码验证一下：

```python
import numpy as np
f1 = np.float32(123456789)
f2 = np.float32(123456791)
print(f1 == f2)
```

输出是True。123456789 和 123456791，在 FP32 看来是同一个数。

现在回头看 6.75 附近那个 0.00000048 的间距，是不是突然觉得它小得可以忽略了？

这就是浮点数最重要的一个特性：**格点在数轴上不是均匀分布的，越靠近 0 越密集，越远离 0 越稀疏。** 在 6.75 附近，相邻两个值之间只差不到百万分之一；到了一千万附近，间距扩大到了 1；到了一亿附近，间距已经膨胀到了 8。

为什么会这样？回到科学计数法就好理解了。FP32 有 23 位尾数，表示的有效数字始终就是那么多。当数字本身很小的时候，这些有效数字足以覆盖到小数点后很远的地方；当数字本身很大的时候，光是表示整数部分就已经把有效数字用完了，小数部分根本顾不上。

这个“格点不均匀”的画面，是你理解浮点数最关键的一个直觉跃迁。一旦建立了这个画面，后面遇到的所有精度相关的问题，比如梯度太小被“吞掉”、大数加小数小数“消失”、两个接近的数相减精度剧烈损失，全都变成了同一个原理的不同表现。

1.8.8 大数吞掉小数：一亿加一等于一亿

上一节你已经看到了，在一亿附近 FP32 的格点间距是 8。这意味着如果你有一个值是一亿，你给它加上一个小于 8 的数，结果不会有任何变化——那个小数直接被吞掉了。

用代码感受一下：

```python
import torch

a = torch.tensor(1e8, dtype=torch.float32)
b = a + 1
print(a == b)
```

输出是True。一亿加一等于一亿。

这不是 bug，这是浮点数在大数区域“分辨率下降”的必然结果。FP32 只有大约 7 位有效数字，一亿已经是 9 位数了，个位上的变化根本超出了它的分辨能力。

你可能觉得这是一个极端的例子，平时不会遇到。但在大模型训练中，这种“大数吞小数”的场景其实比你想象的常见。当你把几十亿个参数的梯度加在一起的时候，累加结果可能很大，而单个梯度值可能很小，小到被大的累加结果直接吞掉。这是混合精度训练需要用 FP32 来做参数更新的底层原因之一，后面会详细讲到。

1.8.9 溢出与那些特殊的值

每种浮点格式都有自己能表示的最大值和最小值。当一次运算的结果超出了最大值，就叫上溢（overflow），结果会变成一个特殊标记：inf（无穷大）。反过来，当一个非常小的数字小到该格式无法表示时，就叫下溢（underflow），结果可能变成 0。

除了inf，还有一个你在大模型训练中会频繁遇到的特殊值：nan，意思是“不是一个数字”（Not a Number）。它出现在一些数学上没有意义的运算结果中，比如 0 除以 0，或者对负数求平方根。

这两个值有一个让人头疼的特性：传染性。

一旦运算链条中出现了一个inf，后续的运算就有可能产生nan。比如inf乘以 0 在数学上没有意义，结果就是nan。而nan参与后续运算，结果通常还是nan。你可以用一段代码感受一下这个传染过程：

```
import torch

x = torch.tensor(1e38, dtype=torch.float32)
y = x * 100
print(y)

z = y * 0
print(z)

w = z + 1
print(w)
```

第一步，一个很大的数乘以 100，超出了 FP32 的上限，y 变成了inf。第二步，inf乘以 0 得到nan。第三步，nan加 1 还是nan。从这一刻起，你的计算链条上后续很多结果都会被污染。

如果这条计算链是一个神经网络的前向传播，那么从产生inf的那一层开始，后面很多运算都可能被污染，最终 loss 也可能变成nan。这就是开头提到的“模型炸了”的底层机制。

注意一个重要的点：这种崩溃不是随机的、碰运气的事情。它几乎总有一条确定的因果链。通常的路径是这样的：学习率过大，导致某一步参数更新的幅度过大，导致下一轮前向传播中某一层的输出值急剧膨胀，超过了当前浮点格式的上限，变成inf，然后经过一两步运算就变成nan，全面扩散。理解了这个链条，你才能有针对性地排查，而不是毫无方向地反复重启训练碰运气。

1.8.10 FP16 为什么不够用

现在你已经知道了 FP32、FP16、BF16 的区别。接下来一个自然的问题是：训练大模型的时候，到底用哪种？

先回忆一下前面算过的账。一个 8B 模型用 FP32 存储需要大约 32GB，用 BF16 或 FP16 则只需要约 16GB。而且训练过程中不只是存参数，还要存中间计算结果和梯度，实际占用会成倍增长。所以用 16 位格式代替 32 位，存储占用几乎减半，这对于能不能把模型装进你的显卡来说，是生死攸关的差别。

更重要的是，现代 GPU 内部有专门为 16 位浮点运算设计的加速单元，16 位运算

的速度可以是 32 位的两倍甚至更高。

所以用 16 位几乎是必须的。问题在于：FP16 还是 BF16？

FP16 的指数只有 5 位，能表示的最大值只有 65504。在大模型训练中，某一层的中间结果超过这个值的概率并不低。一旦超过，就是上溢，就是`inf`，然后就是一连串的`nan`。而在另一个方向上，FP16 能表示的最小正数也比 FP32 大得多，训练过程中那些数值很小的梯度，在 FP16 下很容易被直接舍入成 0，也就是下溢。一头是大数会炸，一头是小数会消失，FP16 两头都不安全。

为了至少解决下溢的问题，人们发明了一种叫做“损失缩放”（loss scaling）的技术：在反向传播计算梯度之前，先把 loss 乘以一个很大的数（比如 1024），让梯度的数值被整体放大，避免那些很小的梯度下溢变成 0；等梯度算完之后，再除回来。你可以把它想象成用放大镜把一幅很浅的铅笔画放大，看清细节之后再缩回原来的尺寸。

这个方法有用，但增加了复杂度——你需要小心地选择放大倍数，如果放大太多，梯度又会上溢；放大太少，还是下溢。在实际操作中需要动态调整这个倍数，比较麻烦。

BF16 的出现几乎一劳永逸地解决了这个问题。它的指数和 FP32 一样是 8 位，常用数值范围和 FP32 非常接近，上溢的风险也接近 FP32，大多数情况下不需要 loss scaling。代价是精度更低，尾数只有 7 位。但前面说过，神经网络对精度的容忍度很高。在绝大多数大模型预训练任务中，BF16 的精度已经足够了。

不过有一个硬件限制你需要知道：BF16 不是所有 GPU 都支持。比较新的 GPU 有原生的 BF16 硬件加速，但一些较老型号的 GPU 不支持。在不支持的硬件上强行使用 BF16，计算机只能通过软件来模拟，速度反而更慢。如果你的公司用的 GPU 型号比较混杂，这个差异就会成为一个实际的坑。你在写训练脚本的时候指定了 BF16，结果被调度到了一台不支持 BF16 的机器上，性能可能会突然变差。

1.8.11 混合精度：各取所长

你现在可能会想：如果 16 位速度快但精度有限，32 位精度高但慢，有没有办法把两者的优点结合起来？

这正是工业界的做法，叫做混合精度训练（mixed precision training）。核心思路很简单：

模型在前向传播和反向传播的时候，用 16 位（FP16 或 BF16）来计算——因为这些计算量巨大，用低精度可以显著提速、省存储。但在后台，会单独维护一份 FP32 精度的参数副本（叫做“主权重”）。也就是说，模型同时存着两份参数：一份 16 位的用于前向和反向计算，一份 32 位的用于接收梯度更新。每次更新完 32 位的主权重后，再把它转换回 16 位，覆盖掉计算用的那份，进入下一轮训练。

为什么参数更新要用 FP32？因为每一步更新的幅度通常非常小。比如一个参数的值是 1.0，这一步的更新量可能只有 0.00001。如果你的参数存的是 BF16（只有大约 2 到 3 位十进制有效数字），1.0 + 0.00001 在 BF16 下会被舍入成 1.0——这次更新就被完全吞掉了，等于没有更新。你在前面“给二进制串加一”的部分已经见过这种现象了：格点之间是有间距的，小于这个间距的变化就会被吞掉。成千上万步这样的更新被吞掉，模型就停止进步了。但如果参数存的是 FP32（大约 7 位有效数字），1.0 + 0.00001 = 1.00001，更新得以保留。

所以混合精度的分工是：计算密集的部分交给 16 位，速度快；精度敏感的参数更新交给 32 位，不丢细节。两者各司其职。

如果你用的是 FP16 做计算部分，通常还需要配合前面提到的 loss scaling 来防止梯度下溢。如果用 BF16 做计算部分，大部分情况下不需要 loss scaling，用起来更省心。这也是 BF16 在实际训练中更受欢迎的一个原因。

关于混合精度训练在 PyTorch 里怎么写代码、怎么配置，你现在不需要知道。后面涉及训练流程的时候会再接触到它。这里你只需要记住这个核心思想：计算用低精度提速，参数更新用高精度保精度。

实战避坑 · 梯度累积时的精度陷阱

当你的显存不够大、没法用大 batch size 的时候，一种常见的做法是“梯度累积”：跑好几个小 batch，把每次算出来的梯度加起来，攒够了再更新一次参数。问题在于：如果你的梯度是 BF16 格式的，每次累加的时候都会产生舍入误差，累积的次数越多，误差越大。累积 4 次可能没什么感觉，但如果你为了模拟一个很大的 batch size 累积了 32 次甚至 64 次，最后攒出来的梯度和“真正用大 batch 算出来的梯度”之间可能已经差了不少，训练效果会莫名其妙地变差。解决方法是让梯度累积在 FP32 精度下进行。大多数主流训练框架（比如 DeepSpeed、FSDP）的混合精度模式默认就是这么做的，但如果你自己手写训练循环，这个细节很容易漏掉。

1.8.12 加载模型时的一个高频操作

虽然训练流程的细节后面才会讲到，但有一个和浮点格式直接相关的操作，你在入职后的第一周就很可能用到：加载一个别人训练好的模型。

前面算过，一个 8B 模型用 FP32 加载需要大约 32GB，用 BF16 加载只需要约 16GB。如果你的显卡显存只有 24GB，那 FP32 根本装不下，必须用 16 位格式加载。

在使用 Hugging Face 的 Transformers 库（这是大模型领域最主流的工具库之一）加载模型的时候，你可以通过一个参数来指定加载精度：

```
from transformers import AutoModelForCausalLM
import torch

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    torch_dtype=torch.bfloat16
)
```

这里`torch_dtype=torch.bfloat16`就是告诉程序：把模型的所有参数用 BF16 格式加载到显存里。如果你不写这个参数，很多模型默认会用 FP32 加载，占用的显存直接翻倍。

反过来，如果你的 GPU 不支持 BF16，你在这里写了`torch.bfloat16`，模型加载之后做计算可能会出现各种问题，从速度异常到结果全是`nan`。

这是你在日常工作中最高频的浮点格式相关操作之一。每次加载模型之前，想一想：我的显卡支持什么格式？模型本身是用什么格式训练的？我的显存够不够？这三个问题想清楚了，大部分与 dtype 相关的问题就不会出现。

实战避坑 · checkpoint 保存和加载的 dtype 不一致，模型“悄悄变傻”

训练的时候用 BF16，保存 checkpoint 的时候没有注意，有些代码或框架默认会把参数转成 FP32 再存。这本身没问题。但另一个同事加载这个 checkpoint 继续训练的时候，没有手动指定`torch_dtype=torch.bfloat16`，于是模型以 FP32 被加载进来了，显存占用翻倍。他发现显存不够，就把 batch size 砍了一半。训练能跑了，但效果变差了。他花了两天排查数据和学习率的问题，最后才发现根源是 dtype 不对导致显存预算被浪费了一半。

反过来的情况也有：模型是 FP32 训练的，你拿到 checkpoint 之后用 BF16 加载想省显存，精度从 7 位有效数字直接降到 2-3 位，某些对精度敏感的层（比如最后的输出层）可能会出现明显的效果退化，但这种退化不会报错，只会体现在评测指标上，非常隐蔽。建议养成习惯：每次保存和加载 checkpoint 的时候，在日志里打印一下`model.dtype`或者第一个参数的 dtype，确认和预期一致。

1.8.13 关于精确计算的一个补充

前面讲了 0.1 + 0.2 不等于 0.3 的故事。你可能会问：那如果我真的需要精确到分的计算，比如算钱，怎么办？

在涉及金融、会计等场景的时候，通常不会用浮点数，而是用一种叫做定点数或十进制数的方式来存储，Python 里有一个`Decimal`类型专门做这件事。但这些和你做大模型没有任何关系。

在深度学习的世界里，我们不仅不怕浮点数的近似，甚至主动拥抱它。模型的参数是通过大量数据统计出来的近似值，预测结果本身就是概率性的。在这样一个“一切都是近似”的框架里，去追求小数点后第八位的精确，没有任何意义。你需要关心的不是“够不够精确”，而是“精度的损失有没有大到影响训练的稳定性”。前面讲的 FP16 溢出、BF16 精度丢失、混合精度更新被吞掉，全都是在讨论这个边界在哪里。

1.8.14 几件容易搞混的事

在结束这一节之前，有几个初学者特别容易混淆的点，值得单独拿出来说一下。

FP16 和 BF16 不是差不多的东西。它们虽然都是 16 位，但位宽分配策略完全不同。FP16 把更多位给了精度（10 位尾数），BF16 把更多位给了范围（8 位指数）。在大模型预训练阶段，BF16 几乎全面取代了 FP16，因为范围够大、不用操心 loss scaling。但到了推理阶段（用训练好的模型来回答问题），又有另一套量化方案，比如 INT8、INT4，那是另一个话题。总之，“都是 16 位”这个表面上的相似性，掩盖了它们本质上的巨大差异。

不要觉得“位宽越高越好，FP64 肯定更稳”。FP64 确实精度更高，但现在主流的 GPU 对 FP64 的计算能力非常有限，通常只有 FP32 速度的几十分之一。用 FP64 训练大模型，在速度和存储两个维度上都完全不可行。深度学习一个有趣的特性就是：模型对噪声和近似有惊人的容忍度。低精度不仅不是妥协，反而是经过工程验证的、实用且高效的选择。

训练时遇到 NaN 不是“运气不好”。新手遇到训练出 NaN 往往反复重启碰运气。但前面已经讲过，NaN 几乎总有一条从溢出到传染的确定因果链。理解了这条链条，你

就能有针对性地回溯：到底是哪一层的输出首先变成了`inf`？是不是学习率在某个阶段突然跳大了？这种排查方式比重启碰运气高效得多。

同一段代码在不同 GPU 上可能跑出不一样的结果。同一份代码在不同 GPU 型号、不同 CUDA 版本上跑出的结果可能在小数点后几位就不同。这是因为不同硬件在做并行计算的时候，浮点加法的累加顺序可能不一样，而浮点加法不满足结合律（也就是说，(a + b) + c 和 a + (b + c) 在浮点数的世界里可能不相等，因为每一步的舍入误差不同）。这不是 bug，是浮点数的本质属性，但会给“复现实验结果”带来困扰。如果你发现自己的运行结果和同事的结果小数点后第四位开始不同，大概率不是你的代码有问题。

面试考点 · 浮点加法不满足结合律

常见问法：“为什么同一份训练代码在不同 GPU 上跑出的结果不完全一样？如何理解浮点数运算的不可复现性？”
典型错误：有人回答“因为随机种子不一样”。随机种子是一个因素，但即使固定了所有随机种子，不同硬件上的浮点计算结果仍然可能不同。
正确思路：随机种子确实要先排查，但它不是唯一原因。浮点加法不满足结合律，GPU 并行归约时会把一大串数拆成不同小组再合并，不同硬件或 CUDA 库的累加顺序可能不一样，最终结果就在小数点后几位出现差异。这种差异通常很小，不影响模型的最终训练效果，但会让“完全复现”别人的实验结果变得几乎不可能。

1.8.15 自检清单

1. 为什么 0.1 在计算机里不能被精确存储？
   0.1 的二进制表示是一个无限循环小数。由于计算机的存储位数有限（如 32 位或 64 位），只能截断保存，这就必然会产生精度丢失，就像十进制中无法用有限的位数写完 1/3 一样。
2. 1 字节等于多少位？一个 FP32 的数占多少字节？一个 8B 参数的模型用 BF16 存储大约占多大空间？
   1 字节等于 8 位。一个 FP32（单精度浮点数）占 32 位，即 4 字节。一个 8B（80 亿）参数的模型，如果用 BF16（16 位，即 2 字节）格式存储，大约需要占据 8B × 2 字节 = 16GB 的空间。
3. 浮点数为什么要拆成符号位、指数和尾数？这三部分分别负责什么？
   符号位决定正负，指数决定数量级，尾数决定这个数量级里的精细位置。这样同样的位数既能表示很小的数，也能表示很大的数，但代价是数轴上的格点不均匀。
4. FP32 的格点在数轴上是均匀分布的吗？在数字 6.75 附近和在一千万附近，相邻两个可表示的值之间的间距差了多少个数量级？
   不是均匀分布的，越靠近 0 格点越密集，数值越大越稀疏。在 6.75 附近，相邻两个数值之间的间距大约是百万分之一；而在接近一千万时，间距会放大到 1 左右，两者差了六个数量级。
5. BF16 为什么在大模型训练中比 FP16 更受欢迎？
   因为 BF16 保留了和 FP32 相同的 8 位指数位，只是牺牲了尾数位。虽然它的精度比 FP16 更粗糙，但常用数值范围和 FP32 非常接近，在大模型训练中不容易发生梯度下溢或上溢，数值稳定性更好。

6. 混合精度训练的核心思想是什么？
   为了兼顾训练速度与模型精度。在计算量大的前向和反向传播中，使用 16 位浮点数以加快计算速度并节省显存；而在更新模型权重时，保留一份 FP32 的主权重来累加微小的梯度，防止因精度不足导致更新失效。

1.8.16 Python 交互实验室

正文中我们用固定的例子讲解了浮点数的二进制表示、格点间距和溢出传染，但固定的例子无法让你自由探索“不同数值下会发生什么”。配套资源包中的 `float_precision_demo.py` 是一个交互式实验台。运行它，你会进入五个实验模块，每个模块内你可以反复输入不同数值，实时观察输出变化。例如在“大数吞小数”实验中，固定大数为 1e8，把小数从 1 逐渐减小到 0.1，输出会显示加法从有效变为无效的临界点；在“十进制小数转二进制”实验中，输入 0.5 和 0.1 对比，你会看到为什么前者能精确存储而后者不能。每个实验结束后可以按回车继续尝试新数值，或输入 n 进入下一个实验。

1.9 显卡与 CUDA 常识：你的算力底座

1.9.1 你即将进入的燃烧室

在深度学习这个行业里，一直有一个说法叫“三驾马车”：数据、算法、算力。数据是原材料，算法是方法论，而算力，就是你把数据和算法变成一个真正能用的模型所需要的计算资源。

前面几节，你已经学会了怎么连上远程服务器，怎么管理 Python 环境，怎么用 PyTorch 搭一个最简单的模型。到目前为止，一切都还比较轻量。但当你真正入职、开始接触大模型的训练和推理，你会发现一个东西会反复出现在你的日常工作中——不是某个算法、不是某个框架，而是一块硬件：**显卡**，也就是 GPU。

你会在组会上听到同事说“这个实验需要 8 张 A100”。你会在群里看到有人抱怨“4090 的显存不够，跑不了 70B 的模型”。你会在配环境的时候遇到一种让人抓狂的报错，折腾两个小时才发现是 CUDA 版本不对。你甚至会听到老板在算一笔账：“这个集群一个月光电费就要三十万。”

这些场景里出现的 A100、显存、CUDA、集群这些词，你现在可能还不太清楚它们到底是什么。没关系，这一节就是专门帮你把这些东西理清楚的。我不会让你写任何 GPU 底层代码，也不会扔给你一堆芯片架构图。你需要的是一张清晰的认知地图：GPU 是什么、为什么需要它、它的核心约束是什么、以及当你在工作中遇到 GPU 相关的问题时，你至少知道该往哪个方向排查。

1.9.2 CPU 和 GPU：博士与小学生

先回答一个最基本的问题：你的电脑里明明有一颗很强的 CPU，为什么训练模型非要用 GPU？

这要从两种芯片截然不同的设计思路说起。

CPU 的设计目标是“什么都能干”。你的操作系统在靠它运行，你的浏览器在靠它跑，你同时开着微信、VS Code 和十几个浏览器标签页，CPU 要负责在这些任务之间灵活调度，处理各种复杂的判断和跳转逻辑。为了完成这些高难度的工作，CPU 的